4
Sorting

Sorting is the most fundamental algorithmic problem in computer science and a rich source of programming problems for two distinct reasons. First, sorting is a useful operation which efficiently solves many tasks that every programmer encounters. As soon as you recognize your job is a special case of sorting, proper use of library routines make short work of the problem.

Second, literally dozens of different sorting algorithms have been developed, each of which rests on a particular clever idea or observation. Most algorithm design paradigms lead to interesting sorting algorithms, including divide-and-conquer, randomization, incremental insertion, and advanced data structures. Many interesting programming/mathematical problems follow from properties of these algorithms.

In this chapter, we will review the primary applications of sorting, as well as the theory behind the most important algorithms. Finally, we will describe the sorting library routines provided by all modern programming languages, and show how to use them on a non-trivial problem.

4.1 Sorting Applications

The key to understanding sorting is seeing how it can be used to solve many important programming tasks:

- *Uniqueness Testing* — How can we test if the elements of a given collection of items S are all distinct? Sort them into either increasing or decreasing order so that any repeated items will fall next to each other. One pass through the elements testing if $S[i] = S[i+1]$ for any $1 \leq i < n$ then finishes the job.

- *Deleting Duplicates* — How can we remove all but one copy of any repeated elements in S? Sort and sweep again does the job. Note that the sweeping is best done by maintaining two indices — *back*, pointing to the last element in the cleaned-out prefix array, and i, pointing to the next element to be considered. If $S[back] <> S[i]$, increment *back* and copy $S[i]$ to $S[back]$.
- *Prioritizing Events* — Suppose we are given a set of jobs to do, each with its own deadline. Sorting the items according to the deadline date (or some related criteria) puts the jobs in the right order to process them. Priority queue data structures are useful for maintaining calendars or schedules when there are insertions and deletions, but sorting does the job if the set of events does not change during execution.
- *Median/Selection* — Suppose we want to find the kth largest item in set S. After sorting the items in increasing order, this fellow sits in location $S[k]$. This approach can be used to find (in a slightly inefficient manner) the smallest, largest, and median elements as special cases.
- *Frequency Counting* — Which is the most frequently occurring element in S, i.e., the mode? After sorting, a linear sweep lets us count the number of times each element occurs.
- *Reconstructing the Original Order* — How can we restore the original arrangement of a set of items after we permute them for some application? Add an extra field to the data record for the item, such that the ith record sets this field to i. Carry this field along whenever you move the record, and later sort on it when you want the initial order back.
- *Set Intersection/Union* — How can we intersect or union the elements of two containers? If both of them have been sorted, we can merge them by repeatedly taking the smaller of the two head elements, placing them into the new set if desired, and then deleting the head from the appropriate list.
- *Finding a Target Pair* — How can we test whether there are two integers $x, y \in S$ such that $x + y = z$ for some target z? Instead of testing all possible pairs, sort the numbers in increasing order and sweep. As $S[i]$ increases with i, its possible partner j such that $S[j] = z - S[i]$ decreases. Thus decreasing j appropriately as i increases gives a nice solution.
- *Efficient Searching* — How can we efficiently test whether element s is in set S? Sure, ordering a set so as to permit efficient binary search queries is perhaps the most common application of sorting. Just don't forget all the others!

4.2 Sorting Algorithms

You have quite possibly seen a dozen or more different algorithms for sorting data. Do you remember bubblesort, insertion sort, selection sort, heapsort, mergesort, quicksort,

radix sort, distribution/bin sort, Shell sort, in-order tree traversal, and sorting networks? Most likely your eyes started to glaze by the time you made it halfway through the list. Who needs to know so many ways to do the same thing, especially when there already exists a sorting library function included with your favorite programming language?

The real reason to study sorting algorithms is that the *ideas* behind them reappear as the ideas behind algorithms for many other problems. Understand that heapsort is *really* about data structures, that quicksort is *really* about randomization, and that mergesort is *really* about divide-and-conquer, and you have a wide range of algorithmic tools to work with.

We review a few particularly instructive algorithms below. Be sure to note what useful properties (such as minimizing data movement) come with each algorithm.

- *Selection Sort* — This algorithm splits the input array into sorted and unsorted parts, and with each iteration finds the smallest element remaining in the unsorted region and moves it to the end of the sorted region:

```
selection_sort(int s[], int n)
{
        int i,j;                /* counters */
        int min;                /* index of minimum */

        for (i=0; i<n; i++) {
                min=i;
                for (j=i+1; j<n; j++)
                        if (s[j] < s[min]) min=j;
                swap(&s[i],&s[min]);
        }
}
```

  Selection sort makes a lot of comparisons, but is quite efficient if all we count are the *number* of data moves. Only $n-1$ swaps are performed by the algorithm, which is necessary in the worst case; think about sorting a reversed permutation. It also provides an example of the power of advanced data structures. Using an efficient priority queue to maintain the unsorted portion of the array suddenly turns $O(n^2)$ selection sort into $O(n \lg n)$ heapsort!

- *Insertion Sort* — This algorithm also maintains sorted and unsorted regions of the array. In each iteration, the next unsorted element moves up to its appropriate position in the sorted region:

```
insertion_sort(int s[], int n)
{
        int i,j;                /* counters */

        for (i=1; i<n; i++) {
                j=i;
                while ((j>0) && (s[j] < s[j-1])) {
```

```
                        swap(&s[j],&s[j-1]);
                        j = j-1;
                }
        }
}
```

Insertion sort is particularly significant as the algorithm which minimizes the *amount* of data movement. An *inversion* in a permutation p is a pair of elements which are out of order, i.e., an i, j such that $i < j$ yet $p[i] > p[j]$. Each swap in insertion sort erases exactly one inversion, and no element is otherwise moved, so the number of swaps equals the number of inversions. Since an almost-sorted permutation has few inversions, insertion sort can be very effective on such data.

- *Quicksort* — This algorithm reduces the job of sorting one big array into the job of sorting two smaller arrays by performing a *partition* step. The partition separates the array into those elements that are less than the pivot/divider element, and those which are strictly greater than this pivot/divider element. Because no element need ever move out of its region after the partition, each subarray can be sorted independently. To facilitate sorting subarrays, the arguments to `quicksort` include the indices of the first (`l`) and last (`h`) elements in the subarray.

```
quicksort(int s[], int l, int h)
{
        int p;                  /* index of partition */

        if ((h-l)>0) {
                p = partition(s,l,h);
                quicksort(s,l,p-1);
                quicksort(s,p+1,h);
        }
}

int partition(int s[], int l, int h)
{
        int i;                  /* counter */
        int p;                  /* pivot element index */
        int firsthigh;          /* divider position for pivot */

        p = h;
        firsthigh = l;
        for (i=l; i<h; i++)
                if (s[i] < s[p]) {
                        swap(&s[i],&s[firsthigh]);
                        firsthigh ++;
                }
        swap(&s[p],&s[firsthigh]);
```

```
        return(firsthigh);
}
```

Quicksort is interesting for several reasons. When implemented properly, it is the fastest in-memory sorting algorithm. It is a beautiful example of the power of recursion. The `partition` algorithm is useful for many tasks in its own right. For example, how might you separate an array containing just 0's and 1's into one run of each symbol?

4.3 Program Design Example: Rating the Field

Pretty Polly has no shortage of gentlemen suitors who come a' courting. Indeed, her biggest problem is keeping track of who the best ones are. She is smart enough to realize that a program which ranks the men from most to least desirable would simplify her life. She is also persuasive enough to have talked you into writing the program.

Polly really likes to dance, and has determined the optimal partner height is 180 centimeters tall. Her first criteria is finding someone who is as close as possible to this height; whether they are a little taller or shorter doesn't matter. Among all candidates of the same height, she wants someone as close as possible to 75 kilograms without going over. If all equal-height candidates are over this limit, she will take the lightest of the bunch. If two or more people are identical by all these characteristics, sort them by last name, then by first name if it is necessary to break the tie.

Polly is only interested in seeing the candidates ranked by name, so the input file:

```
George Bush             195     110
Harry Truman            180     75
Bill Clinton            180     75
John Kennedy            180     65
Ronald Reagan           165     110
Richard Nixon           170     70
Jimmy Carter            180     77
```

yields the following output:

```
Clinton, Bill
Truman, Harry
Kennedy, John
Carter, Jimmy
Nixon, Richard
Bush, George
Reagan, Ronald
```

Solution starts below

The key to this problem is sorting under a fairly complex criteria defined over multiple fields. There are at least two different ways we can do this. The first makes multiple sorting passes, sorting first on the *least* important key, then the next least important key, and so on until we finally sort on the major key.

Why do we sort in this order? The minor keys are only used to break ties among the major key ordering. Provided our sorting algorithm is *stable*, meaning it preserves the relative order of equal keys, our work on the minor keys remains intact if it is relevant to the final answer.

Not all sorting algorithms are stable: indeed most fast ones are not! The `insertion_sort` and `selection_sort` functions from Section 4.2 are stable, while `quicksort` is not stable. Before you assume the stability of any sorting function, check the documentation carefully.

The other approach, which is what we opted for, rolls all the keys up into one complex comparison function. Doing it this way made it easiest to take advantage of the C library sorting routine, described in the next section.

4.4 Sorting Library Functions

Whenever possible, take advantage of the built-in sorting/searching libraries in your favorite programming language:

Sorting and Searching in C

The `stdlib.h` contains library functions for sorting and searching. For sorting, there is the function `qsort`:

```
#include <stdlib.h>

void qsort(void *base, size_t nel, size_t width,
          int (*compare) (const void *, const void *));
```

The key to using `qsort` is realizing what its arguments do. It sorts the first `nel` elements of an array (pointed to by `base`), where each element is `width`-bytes long. Thus we can sort arrays of 1-byte characters, 4-byte integers, or 100-byte records, all by changing the value of `width`.

The ultimate desired order is determined by the function `compare`. It takes as arguments pointers to two `width`-byte elements, and returns a negative number if the first belongs before the second in sorted order, a positive number if the second belongs before the first, or zero if they are the same.

Here is a comparison function for sorting integers in increasing order:

```
int intcompare(int *i, int *j)
{
    if (*i > *j) return (1);
    if (*i < *j) return (-1);
```

```
    return (0);
}
```

This comparison function can be used to sort an array `a`, of which the first `cnt` elements are occupied, as follows:

```
qsort((char *) a, cnt, sizeof(int), intcompare);
```

A more sophisticated example of `qsort` in action appears in Section 4.5. The name `qsort` suggests that quicksort is the algorithm implemented in this library function, although this is usually irrelevant to the user.

Note that `qsort` destroys the contents of the original array, so if you need to restore the original order, make a copy or add an extra field to the record as described in Section 4.1.

Binary search is an amazingly tricky algorithm to implement correctly under pressure. The best solution is not to try, since the `stdlib.h` library contains an implementation called `bsearch()`. Except for the search key, the arguments are the same as for `qsort`. To search in the previously sorted array, try

```
bsearch(key, (char *) a, cnt, sizeof(int), intcompare);
```

Sorting and Searching in C++

The C++ Standard Template Library (STL), discussed in Section 2.2.1, includes methods for sorting, searching, and more. Serious C++ users should get familiar with STL.

To sort with STL, we can either use the default comparison (e.g., $\leq$) function defined for the class, or override it with a special-purpose comparison function `op`:

```
void sort(RandomAccessIterator bg, RandomAccessIterator end)
void sort(RandomAccessIterator bg, RandomAccessIterator end,
          BinaryPredicate op)
```

STL also provides a stable sorting routine, where keys of equal value are guaranteed to remain in the same relative order. This can be useful if we are sorting by multiple criteria:

```
void stable_sort(RandomAccessIterator bg, RandomAccessIterator end)
void stable_sort(RandomAccessIterator bg, RandomAccessIterator end,
          BinaryPredicate op)
```

Other STL functions implement some of the applications of sorting described in Section 4.1, including,

- `nth_element` – Return the nth largest item in the container.
- `set_union`, `set_intersection`, `set_difference` – Construct the union, intersection, and set difference of two containers.
- `unique` – Remove all consecutive duplicates.

Sorting and Searching in Java

The `java.util.Arrays` class contains various methods for sorting and searching. In particular,

```
static void sort(Object[] a)
static void sort(Object[] a, Comparator c)
```

sorts the specified array of objects into ascending order using either the natural ordering of its elements or a specific comparator c. Stable sorts are also available.

Methods for searching a sorted array for a specified object using either the natural comparison function or a new comparator c are also provided:

```
binarySearch(Object[] a, Object key)
binarySearch(Object[] a, Object key, Comparator c)
```

4.5 Rating the Field

Our solution to Polly's dating difficulties revolved around making the multi-criteria sorting step as simple as possible. First, we had to set up the basic data structures:

```
#include <stdio.h>
#include <string.h>

#define NAMELENGTH      30              /* maximum length of name */
#define NSUITORS        100             /* maximum number of suitors */

#define BESTHEIGHT      180             /* best height in centimeters */
#define BESTWEIGHT      75              /* best weight in kilograms */

typedef struct {
        char first[NAMELENGTH];         /* suitor's first name */
        char last[NAMELENGTH];          /* suitor's last name */
        int height;                     /* suitor's height */
        int weight;                     /* suitor's weight */
} suitor;

suitor suitors[NSUITORS];               /* database of suitors */
int nsuitors;                           /* number of suitors */
```

Then we had to read the input. Note that we did not store each fellow's actual height and weight! Polly's rating criteria for heights and weights were quite fussy, revolving around how these quantities compare to a reference height/weight instead of a usual linear order (i.e., increasing or decreasing). Instead, we altered each height and weight appropriately so the quantities were linearly ordered by desirability: